OutOfRole in Plain Words

The Debugging Table · IBM Z Datathon 2026 · AI Secured: Innovation Without Exposure

In one paragraph

OutOfRole compares each employee's recent activity with their own normal weeks, and gives security analysts a short daily review list of 20 names. For each person on the list, it gathers the evidence that is new to that person and rare in the company. IBM Granite, running on the same LinuxONE server, turns that evidence into a short explanation, and code checks every number before anyone sees it. Separately, a Netflix-style recommender predicts which websites each person will use next, so routine access can be pre-approved and unused access can be reviewed for removal. Nothing leaves the server, and people appear only as pseudonyms.

The problem

Some people misuse access they legitimately have. They copy files to a USB stick before quitting, upload documents to a leak site, or log into colleagues' computers to snoop. Their logins are valid, so rule-based alerts either miss them or bury analysts in thousands of hits.

What OutOfRole does

1. **Watch.** Every day, compare each person with their own normal weeks. When the last 7 days beat every normal week on some activity (USB copies, after-hours logins, file copies, new computers), their score rises. The 20 highest scores form the day's review list.
2. **Explain.** For each name on the list, rank what they did: new to this person first, then rarest in the company. Granite writes a case file from the top items and must cite each one. Code checks every number and date, and flags weak evidence.
3. **Prevent.** The recommender learns which websites each person uses and predicts the ones they will want next. Those can be pre-approved; anything else goes to a reviewer. Access a person holds but stops using becomes a removal suggestion. Sites no one in the company visited during the training months are blocked.

What we measured

The data is the CERT insider-threat benchmark, in two releases. r4.2 has 1,000 staff and 70 insiders; r5.2 has 2,000 staff and 99 insiders. Settings were frozen on r4.2, then r5.2 was run unchanged. Every number comes from one program, `outofrole_minimal.py`.

Step	Result	r4.2	r5.2
Watch	Insiders on the daily list before their last malicious action	59 of 70	**80 of 99**
	Same, standard 28-day baseline at the same list churn	59 of 70	40 of 99
	Share of each insider's malicious actions on later days (all insiders, missed = 0)	54%	48%
	Normal staff ever placed on the list, over the whole test period	230 of 930	291 of 1,901
Explain	Insiders caught in time whose top 3 evidence items include a real malicious action	66%	85%
	Same, ranking by rarity alone	49%	51%

Step	Result	r4.2	r5.2
	Case files with strong evidence: insiders vs normal staff (no labels used)	53% vs 18%	72% vs 30%
Prevent	Next new websites caught by the recommender's top 20 (popularity in brackets)	21% (14.5%)	19% (13%)
	Access requests per person per month: own history only, then with the recommender	2.31 to 1.82	3.15 to 2.55
	Held sites never used again (removal suggestions)	6.0%	3.6%
	Wikileaks and keylogger-site visits blocked by the new-to-company rule	100%	100%

What this means:

- **On the release we held back for testing (r5.2), the daily list catches twice as many insiders as the standard baseline,** at the same analyst workload. On r4.2 the two tie.
- **The recommender is not the detector.** It predicts which websites people will use next better than popularity does, and that is the job it does well.

How simple it is

Spark turns the raw logs into three tables. Everything else is two short Python files:

- `outofrole_minimal.py` (about 210 lines, about 20 seconds per release on a laptop): the daily list, the evidence ranking, the recommender, the access suggestions and the block rule.
- `case_file.py` (61 lines): the evidence pack, the Granite prompt, the number checker and a plain fallback explanation. A self-test shows that it rejects a made-up fact, a wrong percentage and an uncited claim.

Granite and a small dashboard sit on top.

How it connects to our proposal

We promised	We deliver
Catch insiders with valid logins without flooding analysts	A 20-name daily list
"Learns what each employee normally uses"	Each person's own normal weeks; a recommender learns their websites
"Access it would never suggest is flagged"	Recommended access is pre-approved; anything else goes to review; never-seen sites are blocked
"An AI on the same server explains why"	Granite case files from checked evidence, on LinuxONE
Analysts: a short, ranked list of explained alerts	The daily list plus case files
Compliance: a record of who accessed what and why	Each case file cites dated evidence items
Access admins: permissions held but never used	Removal suggestions (6.0% and 3.6% of held sites) and pre-approval suggestions
Data never leaves the platform	One server, pseudonyms only
Spark matrix, ALS on early months, later months replayed as a live stream	Exactly that: the day-by-day replay is the live stream

What changed: our tests showed that each person's own history catches insiders better than the recommender does. So the recommender moved from detection to access advice, where it measurably helps. The computer and file recommenders were dropped, because they added nothing.

What we say openly

- CERT is synthetic data. These are replay results, not a live deployment.

- The method behind the daily list was chosen after seeing results. The method we had named in advance also beat the baseline (54 vs 28 of 99).
- Evidence is graded on the day each insider was caught in time. 44% (r4.2) and 31% (r5.2) of insiders were first listed before their labelled attack began, when there was nothing malicious to cite yet.
- Two-day sysadmin sabotage is mostly missed.
- Granite and the LinuxONE run are built at the event. The code they plug into is tested here.

Every number above, with the script that produced it, is in `feasibility/Verified_Numbers.md`.